Name: Arslan Ahmad Khan.

Class: MAD Section B

Lecturer: Mam Sadaf

Assignment 01

Question 1: Software Requirements Specification (SRS)

1. Introduction

1.1. Purpose

This document defines the complete functional and non-functional requirements for the **Restaurant App MVP**, a frontend-only React Native mobile application designed to streamline customer dining, table reservations, and order management, while providing restaurant managers with a real-time operational dashboard.

1.2. Scope

- **In Scope:**
    - User authentication (Customer & Manager roles) with local mock validation.
    - Menu browsing with categories, daily specials, search debouncing, sorting, and availability indicators.
    - Cart management with item quantities, special instructions, and promo code discounts.
    - Table reservation system with party size checking, hourly time slots, date validation, and confirmation modals.
    - Order placement supporting Dine-in and Takeaway options.
    - Real-time simulated Order Tracking with step progression timers.
    - Manager Dashboard for managing incoming orders, table reservations, and menu CRUD operations.
    - Global state management via Context API and Reducers, supporting light/dark theme toggling and local persistence using AsyncStorage.
- **Out of Scope:**

- Real backend servers, REST API hosting, and database persistence (MongoDB/PostgreSQL).
- Real payment gateway integration (Stripe, JazzCash, EasyPaisa API integration).
- Native push notifications and SMS gateways.

1.3. Definitions and Acronyms

1. **SRS:** Software Requirements Specification – A document that sets out functional and non-functional requirements of a software system.
2. **MVP:** Minimum Viable Product – A version of a product with just enough features to be usable by early customers who can then feed back product refinement.
3. **UML:** Unified Modeling Language – A general-purpose, developmental modeling language in the field of software engineering.
4. **Hook:** A React function that lets functional components hook into React state and lifecycle features without writing classes.
5. **Context:** A React mechanism that allows data to be passed deeply through the component tree without prop drilling.
6. **Reducer:** A pure function that takes the current state and an action, and returns the next state ((state, action) => newState).
7. **Mock Data:** Hardcoded static datasets structured as JSON or JavaScript modules used to simulate backend responses in the absence of a server.
8. **FlatList:** A high-performance React Native component for rendering performant, scrollable lists of data with lazy rendering

2. Overall Description

2.1. User Roles

Role	Key Permissions & Goals
Customer	Browse daily specials and full menus, search/filter items, manage cart, apply promo codes, reserve tables, place dine-in/takeaway orders, and track order status in real time.
Manager	Oversee incoming customer orders, update order fulfillment stages manually, accept/decline table reservations, and add/edit menu items, prices, and stock availability.

2.2. User Stories

1. **As a** customer, **I want** to browse daily specials on the home screen, **so that** I can quickly order popular items without scrolling through the entire menu. *(Customer Interview)*
2. **As a** customer, **I want** to filter menu items by categories (Starters, Mains, Desserts, Drinks), **so that** I can easily find what I am craving. *(Customer Interview)*
3. **As a** customer, **I want** to book a table for a specific party size and time slot, **so that** my table is ready upon arrival. *(Customer Interview)*
4. **As a** customer, **I want** to apply promo codes (e.g., WELCOME10), **so that** I can receive discounts on my total bill. *(Customer Interview)*
5. **As a** customer, **I want** to track my order progress (Pending → Preparing → Ready → Served) in real time, **so that** I know when my food will arrive. *(Customer Interview)*
6. **As a** customer, **I want** to toggle between light and dark mode themes, **so that** the app is comfortable to view in low-light environments.
7. **As a** customer, **I want** to search for dishes instantly with debounced inputs, **so that** the UI remains responsive while typing.
8. **As a** customer, **I want** to add custom instructions to my cart items (e.g., "no onions"), **so that** my dietary preferences are communicated accurately.
9. **As a** customer, **I want** to view my past and active reservations, **so that** I can cancel them if my plans change.

10. **As a** manager, **I want** to view all incoming customer orders in a centralized dashboard, **so that** I can eliminate phone-in order errors. *(Manager Goal)*
11. **As a** manager, **I want** to manually update order statuses (Preparing, Ready, Served), **so that** kitchen staff and customers stay synchronized. *(Manager Goal)*
12. **As a** manager, **I want** to toggle menu item availability or update prices instantly, **so that** sold-out items cannot be ordered. *(Manager Goal)*

3. Functional Requirements

- **FR-01:** The system shall authenticate users against local mock credentials, validating email format and password length (≥ 8 characters with at least one digit).
- **FR-02:** The system shall switch between Login and Signup modes dynamically via a single toggle state.
- **FR-03:** The system shall restrict access to the Manager Dashboard tab strictly to users with the manager role.
- **FR-04:** The system shall fetch the restaurant menu asynchronously with a simulated 1.5-second network delay, displaying a loading spinner and error retry state upon failure.
- **FR-05:** The system shall render menu items inside a performant FlatList with key extractors and daily special badges.
- **FR-06:** The system shall grey out unavailable menu items and disable their respective "Add to Cart" buttons.
- **FR-07:** The system shall implement pull-to-refresh functionality on the menu browsing screen.
- **FR-08:** The system shall update the screen header title dynamically to reflect the count of currently displayed filtered items.
- **FR-09:** The system shall provide a search bar with a manual debounce of 400 milliseconds using useRef and useDebounce.
- **FR-10:** The system shall display a floating "Back to top" button when the user scrolls down more than 300 pixels.
- **FR-11:** The system shall maintain a history of the last five unique search queries and display them as tappable suggestions when the search input is focused and empty.

- **FR-12:** The system shall manage cart state using a pure reducer supporting actions: ADD_ITEM, REMOVE_ITEM, INCREMENT, DECREMENT, UPDATE_NOTE, CLEAR_CART, APPLY_PROMO, and REMOVE_PROMO.
- **FR-13:** The system shall validate promotional codes (WELCOME10, FEAST20) and calculate percentage-based discounts accurately.
- **FR-14:** The system shall compute order subtotal, service charge (5%), sales tax (15%), and grand total efficiently using useMemo.
- **FR-15:** The system shall validate table reservation forms, ensuring party size is between 1 and 12, dates are not in the past, and phone numbers match the Pakistani mobile format (03XX-XXXXXXX).
- **FR-16:** The system shall disable hourly time slots (12:00 to 22:00) when no suitable tables with sufficient seating capacity are available.
- **FR-17:** The system shall support order placement for either Dine-in (with table assignment) or Takeaway (with pickup time).
- **FR-18:** The system shall automatically transition order status through timers (Pending → Preparing after 10s → Ready after 20s → Served after 30s) on the Order Tracking screen.
- **FR-19:** The system shall allow managers to accept or decline table reservations directly from the dashboard.
- **FR-20:** The system shall persist orders, reservations, and menu changes locally using AsyncStorage to prevent data loss on app restart.

4. Non-Functional Requirements

- **Usability:** The UI adheres to clean mobile design patterns with consistent color palettes, clear validation error messages, and immediate visual feedback on button presses.
- **Performance:** Screen transitions execute at 60 FPS. List rendering leverages FlatList virtualization, and calculations are memoized using useMemo and useCallback to prevent unnecessary re-renders.
- **Responsiveness:** Layouts utilize flexible Flexbox styling (flex: 1, percentage dimensions) to scale smoothly across various mobile screen sizes and aspect ratios.

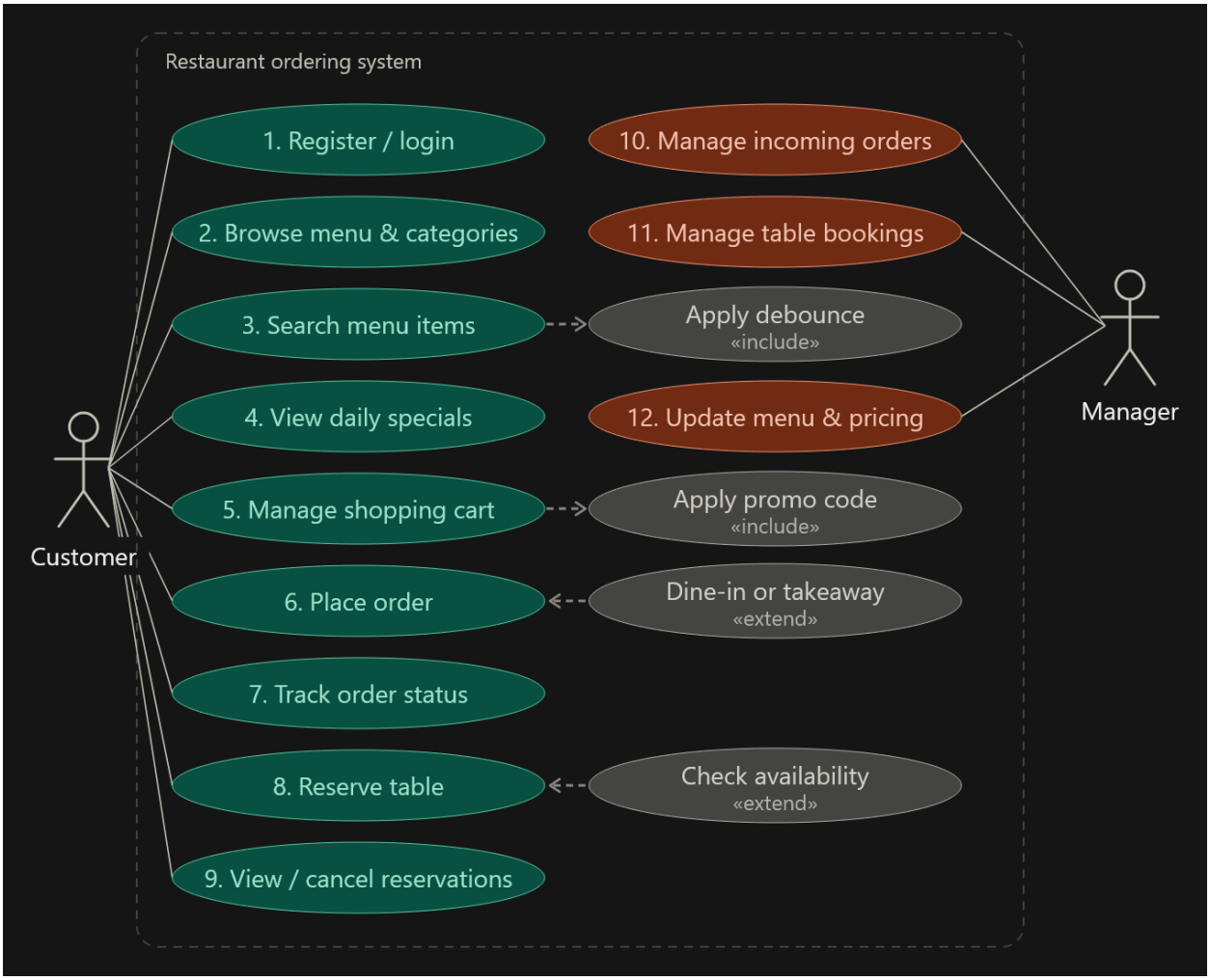
- **Maintainability:** Codebase is cleanly organized into modular directories (components/, screens/, context/, reducers/, hooks/, data/, navigation/). Reusable logic is extracted into custom hooks.
- **Data Handling:** In the complete absence of a backend server, local datasets are structured in modular JS files and managed via React Context and Reducers, with persistent fallback via AsyncStorage.

5. Client-Side Data Model (Mock Data)

Data Set Name	Fields	Description
Users	id, name, email, password, role	Stores registered customer and manager accounts for authentication.
Categories	id, name, icon	Defines menu categories (Starters, Mains, Desserts, Drinks).
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable	Contains all dishes offered by the restaurant.
Tables	id, tableNumber, capacity, isAvailable	Represents physical dining tables available for reservation.
Reservations	id, userId, tableId, date, timeSlot, partySize, status	Tracks table bookings made by customers.
Orders	id, userId, items, total, type, status, timestamp, tableId	Stores active and past customer orders.

Question 2: UML Diagrams Specifications

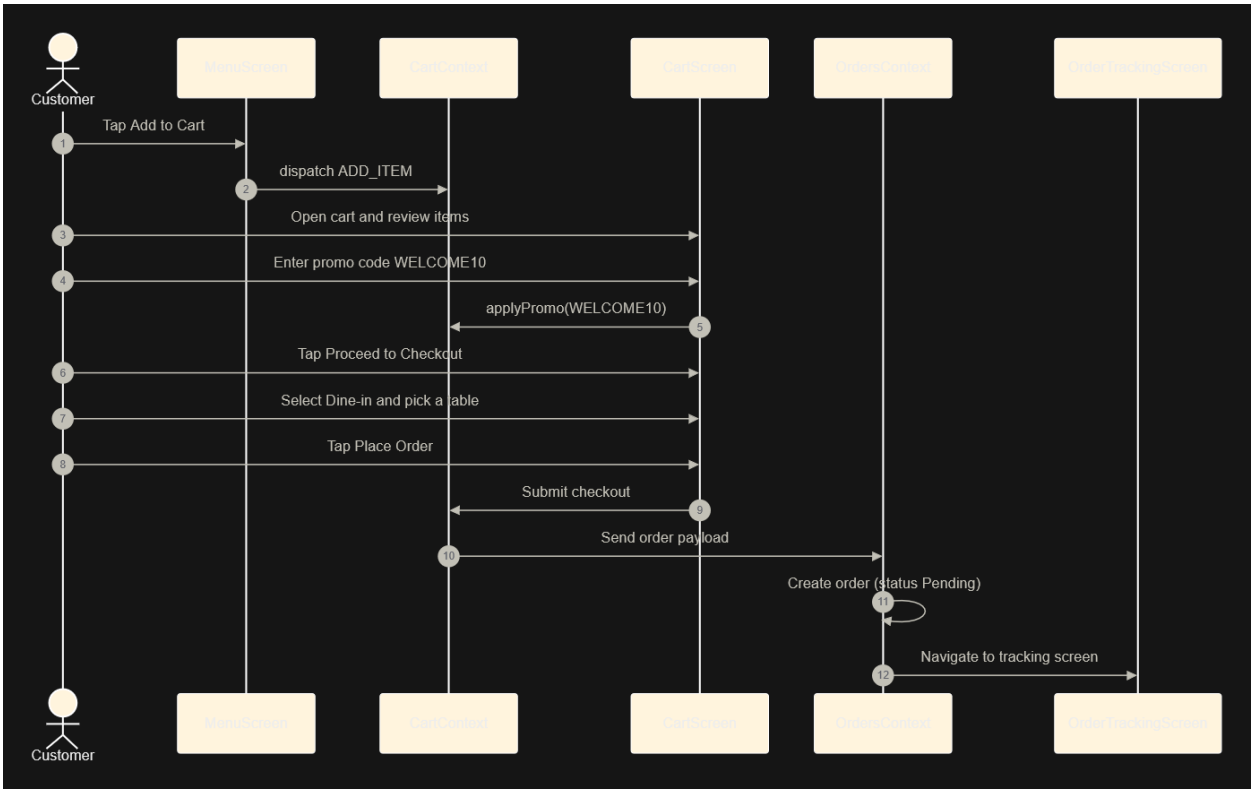
1. Use Case Diagram



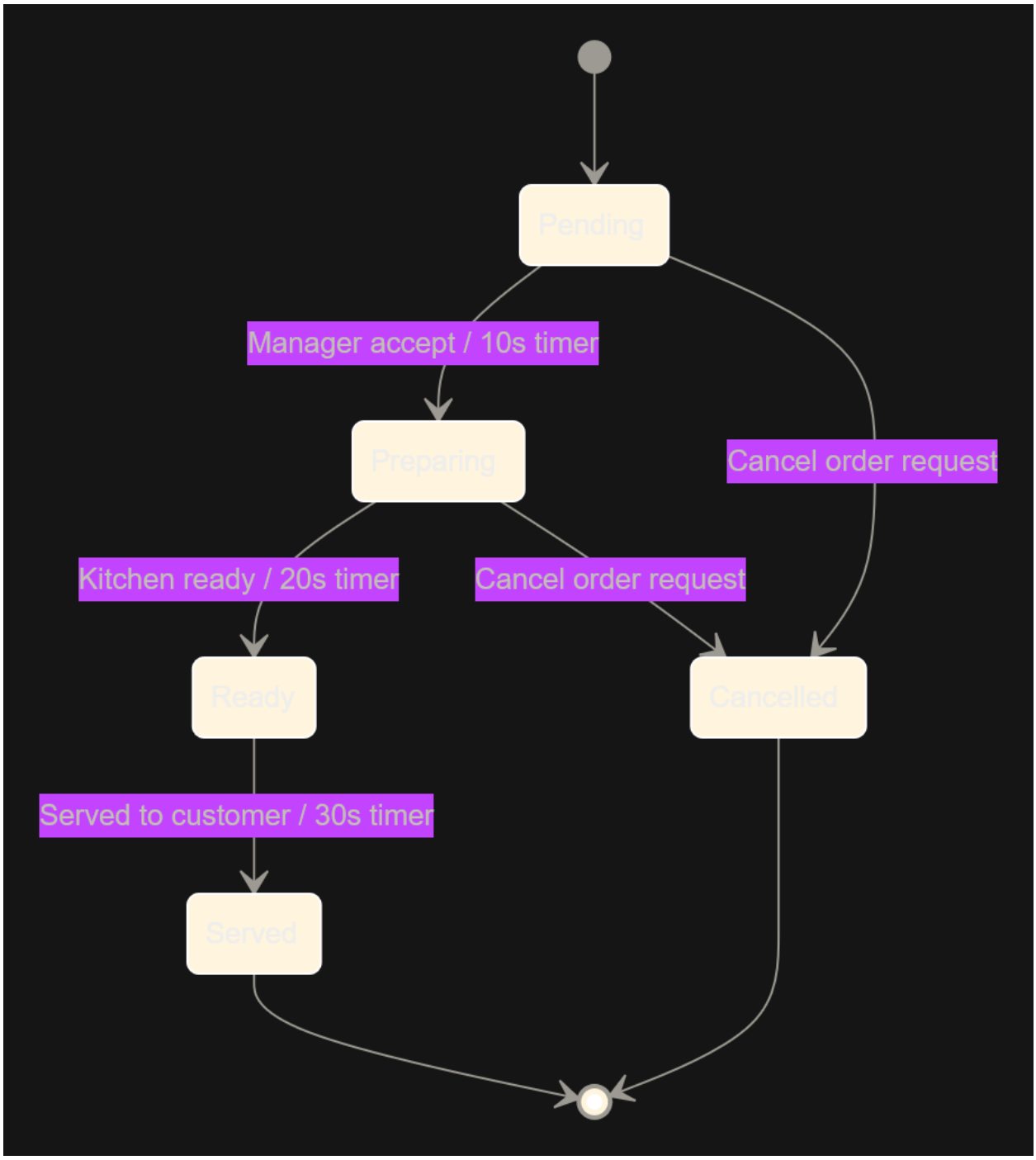
2. Class Diagram



3. Sequence Diagram (Customer Places Order)



4. State Machine Diagram (Order Lifecycle)



5. Component Diagram

